

JARA

Advanced To Do List

Panduan Lengkap Pengerjaan Tim 4 Orang
Laravel 13 - Database - GitHub - Localhost

Target akhir	Aplikasi berjalan di localhost dan source code tersimpan di GitHub
Stack	Laravel 13.x, Blade, database SQLite/MySQL
Tim	4 orang dengan feature branch terpisah
Modul	User/Admin, Task, Project/Team, Priority/Progress

Dokumen kerja kelompok - versi praktis dari setup sampai integrasi akhir.

Daftar Isi

1. Target dan ruang lingkup JARA
2. Pembagian tugas 4 orang
3. Database dan relasi
4. Setup bersama sebelum mulai coding
5. Tugas Orang 1 - User & Admin Management
6. Tugas Orang 2 - Task Management
7. Tugas Orang 3 - Project & Team Collaboration
8. Tugas Orang 4 - Priority, Deadline & Progress
9. Integrasi seluruh branch
10. Menjalankan hasil akhir di localhost
11. Push final ke GitHub dan README
12. Jira board dan pembagian issue
13. Checklist pengujian akhir
14. Troubleshooting umum

Prinsip utama - Semua anggota bekerja dari repository yang sama, tetapi menggunakan branch masing-masing. Jangan coding langsung di main.

1. Target dan Ruang Lingkup JARA

JARA adalah aplikasi advanced to-do list untuk mengelola pekerjaan pribadi maupun tim. Target tugas kelompok adalah membuat aplikasi yang dapat dijalankan secara lokal, memiliki database yang konsisten, dan dapat di-clone dari GitHub oleh anggota lain.

Fitur	Hasil yang diharapkan
Akun pengguna	Register, login, logout, dan role user/admin.
Task	Create, read, edit, delete, dan menandai task selesai.
Project	Task dapat dikelompokkan ke dalam project.
Prioritas & deadline	Task memiliki prioritas dan tenggat waktu.
Kolaborasi	Project memiliki anggota dan task dapat di-assign ke user.
Progress	Menampilkan total task, task selesai, dan persentase progress.
Admin	Admin dapat melihat, menambah, dan menghapus akun pengguna.
GitHub	Semua source code masuk repository dengan riwayat commit/branch yang jelas.

2. Pembagian Tugas 4 Orang

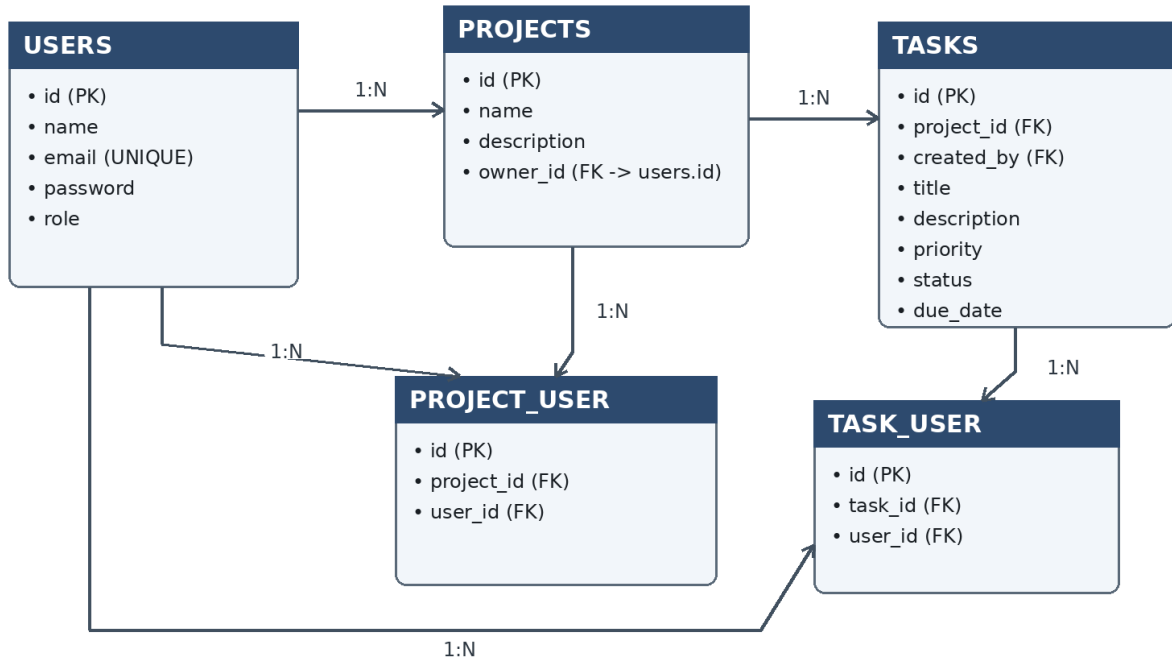
Orang	Branch	Fokus	Output utama
1	feature/user-admin	User & Admin Management	Auth, role admin, halaman kelola user.
2	feature/task	Task Management	CRUD task, status selesai, detail task.
3	feature/project-team	Project & Team Collaboration	CRUD project, anggota project, assign user.
4	feature/progress	Priority, Deadline & Progress	Filter prioritas/deadline, dashboard progress.

Dependensi - Orang 4 membutuhkan data project dan task. Karena itu dashboard final baru diuji setelah branch Orang 2 dan Orang 3 sudah masuk develop. Namun UI/dashboard dapat dikerjakan lebih awal dengan data dummy.

3. Database dan Relasi

Gunakan kontrak database yang sama sejak awal agar branch tidak saling bertabrakan. Lima tabel inti: users, projects, project_user, tasks, task_user.

ERD Sederhana JARA



Gambar 1. ERD JARA

Tabel	Kolom penting	Fungsi
users	id, name, email, password, role	Akun user/admin.
projects	id, name, description, owner_id	Data project dan pemilik.
project_user	project_id, user_id	Anggota project (many-to-many).
tasks	project_id, created_by, title, description, priority, status, due_date	Data pekerjaan.
task_user	task_id, user_id	User yang ditugaskan ke task (many-to-many).

Progress tidak perlu disimpan sebagai kolom. Nilainya dihitung dari jumlah task dengan status done dibagi total task pada project.

$$\text{progress (\%)} = (\text{jumlah task status done} / \text{total task project}) \times 100$$

4. Setup Bersama Sebelum Mulai Coding

Bagian ini dilakukan sekali oleh pemilik repository atau dikerjakan bersama. Setelah base project stabil di develop, barulah setiap anggota membuat branch fitur.

4.1 Pastikan project Laravel berjalan

```
cd C:\Users\Fazl\tugas-laravel
php artisan --version
composer install
npm install
npm run build
php artisan serve
```

Buka <http://127.0.0.1:8000>. Jika halaman Laravel muncul, lanjutkan.

4.2 Buat repository GitHub

15. Buat repository GitHub, contoh nama: JARA-Advanced-ToDoList.
16. Jangan centang pembuatan README bila project lokal sudah memiliki file dan ingin menghindari konflik awal.
17. Di folder project lokal, inisialisasi Git dan hubungkan remote.

```
git init
git add .
git commit -m "chore: initial Laravel project"
git branch -M main
git remote add origin <URL_REPOSITORY_GITHUB>
git push -u origin main
```

4.3 Buat branch develop

```
git checkout -b develop
git push -u origin develop
```

Aturan - main hanya untuk versi stabil. Semua feature branch dibuat dari develop dan merge kembali ke develop melalui Pull Request (PR).

4.4 Buat kontrak migration bersama

Agar migration lintas branch tidak bentrok, base schema dibuat dulu pada develop. Gunakan urutan berikut.

```
php artisan make:migration add_role_to_users_table --table=users
php artisan make:model Project -m
php artisan make:migration create_project_user_table
php artisan make:model Task -m
php artisan make:migration create_task_user_table
```

Isi migration sesuai desain berikut. Tanggal prefix filename boleh berbeda; yang penting urutannya membuat projects sebelum tasks dan tasks sebelum task_user.

```
// add_role_to_users_table
Schema::table('users', function (Blueprint $table) {
    $table->string('role')->default('user');
});

// projects
Schema::create('projects', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->text('description')->nullable();
    $table->foreignId('owner_id')->constrained('users')->cascadeOnDelete();
    $table->timestamps();
});

// project_user
```

```

Schema::create('project_user', function (Blueprint $table) {
    $table->id();
    $table->foreignId('project_id')->constrained()->cascadeOnDelete();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->unique(['project_id', 'user_id']);
});

// tasks
Schema::create('tasks', function (Blueprint $table) {
    $table->id();
    $table->foreignId('project_id')->constrained()->cascadeOnDelete();
    $table->foreignId('created_by')->constrained('users')->cascadeOnDelete();
    $table->string('title');
    $table->text('description')->nullable();
    $table->string('priority')->default('medium'); // low, medium, high
    $table->string('status')->default('todo'); // todo, in_progress, done
    $table->dateTime('due_date')->nullable();
    $table->timestamps();
});

// task_user
Schema::create('task_user', function (Blueprint $table) {
    $table->id();
    $table->foreignId('task_id')->constrained()->cascadeOnDelete();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->unique(['task_id', 'user_id']);
});

```

4.5 Pilih database lokal

Pilihan termudah untuk tugas lokal adalah SQLite. Jika dosen mewajibkan MySQL, gunakan konfigurasi MySQL.

```

# SQLite (.env)
DB_CONNECTION=sqlite

# MySQL (.env) - gunakan jika diwajibkan
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=jara
DB_USERNAME=root
DB_PASSWORD=

php artisan migrate:fresh

```

Jika tidak ada error, commit database contract ke develop sebelum anggota mulai branch.

```

git add .
git commit -m "chore: add JARA database foundation"
git push origin develop

```

4.6 Cara setiap anggota mulai branch

```
git checkout develop
git pull origin develop
git checkout -b feature/NAMA-FITUR
```

Setelah selesai fitur:

```
git status
git add .
git commit -m "feat: deskripsi fitur"
git push -u origin feature/NAMA-FITUR
```

Kemudian buat Pull Request di GitHub dari feature/... ke develop. Minta minimal satu anggota lain review sebelum merge.

5. Tugas Orang 1 - User & Admin Management

Item	Isi
Branch	feature/user-admin
Tujuan	Register, login, logout, role admin, admin list/add/delete user.
Tabel utama	users
Selesai jika	User bisa login dan admin bisa mengelola akun tanpa error.

5.1 Mulai branch

```
git checkout develop
git pull origin develop
git checkout -b feature/user-admin
php artisan make:controller AuthController
php artisan make:controller Admin/UserController
php artisan make:middleware AdminMiddleware
```

5.2 Update model User

Di app/Models/User.php pastikan field role dapat diisi dan buat relasi dasar yang nanti dipakai anggota lain.

```
protected $fillable = [
    'name', 'email', 'password', 'role',
];

public function ownedProjects() {
    return $this->hasMany(Project::class, 'owner_id');
}

public function projects() {
    return $this->belongsToMany(Project::class);
}

public function assignedTasks() {
    return $this->belongsToMany(Task::class);
}
```

5.3 Buat register/login/logout manual

Karena project awal dibuat tanpa starter kit authentication, gunakan Auth facade dan session bawaan Laravel. Controller minimal:

```
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;

public function register(Request $request) {
    $data = $request->validate([
        'name' => ['required', 'max:255'],
        'email' => ['required', 'email', 'unique:users,email'],
        'password' => ['required', 'min:8', 'confirmed'],
    ]);

    $user = User::create([
        'name' => $data['name'],
        'email' => $data['email'],
        'password' => Hash::make($data['password']),
        'role' => 'user',
    ]);

    Auth::login($user);
    return redirect('/dashboard');
```

```

}

public function login(Request $request) {
    $credentials = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (Auth::attempt($credentials)) {
        $request->session()->regenerate();
        return redirect()->intended('/dashboard');
    }

    return back()->withErrors(['email' => 'Email atau password salah.']);
}

public function logout(Request $request) {
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();
    return redirect('/login');
}

```

5.4 Buat route auth

```

Route::middleware('guest')->group(function () {
    Route::get('/register', [AuthController::class, 'showRegister']);
    Route::post('/register', [AuthController::class, 'register']);
    Route::get('/login', [AuthController::class, 'showLogin'])->name('login');
    Route::post('/login', [AuthController::class, 'login']);
});

Route::post('/logout', [AuthController::class, 'logout'])
    ->middleware('auth')->name('logout');

```

5.5 Buat view auth

- resources/views/auth/register.blade.php: form name, email, password, password_confirmation.
- resources/views/auth/login.blade.php: form email dan password.
- Setiap form POST wajib memakai @csrf.
- Tampilkan error dengan @error("field") ... @enderror.
- Buat resources/views/dashboard.blade.php sebagai halaman setelah login.

5.6 Middleware admin

```

// app/Http/Middleware/AdminMiddleware.php
public function handle($request, Closure $next) {
    if (! auth()->check() || auth()->user()->role !== 'admin') {
        abort(403);
    }
    return $next($request);
}

```

Daftarkan alias admin di bootstrap/app.php pada withMiddleware:

```

$middleware->alias([
    'admin' => \App\Http\Middleware\AdminMiddleware::class,
]);

```

5.7 Admin list/add/delete user

- index(): ambil daftar User::latest()->get() dan tampilkan di admin/users/index.blade.php.
- store(): validasi name/email/password/role, hash password, lalu User::create().
- destroy(): cegah admin menghapus akun dirinya sendiri, lalu hapus user lain.
- Route admin dibungkus middleware [auth, admin].

```

Route::middleware(['auth', 'admin'])->prefix('admin')->group(function () {
    Route::get('/users', [AdminUserController::class, 'index']);
    Route::post('/users', [AdminUserController::class, 'store']);
});

```

```
Route::delete('/users/{user}', [AdminController::class, 'destroy']);
});
```

5.8 Buat satu akun admin

Cara cepat untuk lokal: register user biasa, lalu ubah role melalui Tinker.

```
php artisan tinker
$user = App\Models\User::where('email', 'emailkamu@example.com')->first();
$user->role = 'admin';
$user->save();
exit
```

5.9 Pengujian Orang 1

Tes	Expected
Register email baru	Masuk dashboard dan password tersimpan hashed.
Login benar	Berhasil masuk.
Login salah	Muncul error, tidak login.
Akses /admin/users oleh user biasa	403.
Admin tambah user	User baru muncul di daftar.
Admin hapus user lain	User hilang dari daftar.
Logout	Session berakhir dan kembali ke login.

5.10 Commit dan PR

```
git add .
git commit -m "feat: add authentication and admin user management"
git push -u origin feature/user-admin
```

Definition of Done - Tidak ada password plaintext, route admin terlindungi, validasi form aktif, dan semua tes manual di atas lulus.

6. Tugas Orang 2 - Task Management

Item	Isi
Branch	feature/task
Tujuan	CRUD task dan perubahan status todo/in_progress/done.
Tabel utama	tasks dan task_user (sudah dibuat di database foundation).
Selesai jika	Task dapat dibuat, diedit, dihapus, dilihat, dan ditandai selesai.

6.1 Mulai branch dan generate controller

```
git checkout develop
git pull origin develop
git checkout -b feature/task
php artisan make:controller TaskController --resource
```

6.2 Isi model Task

```
protected $fillable = [
    'project_id', 'created_by', 'title', 'description',
    'priority', 'status', 'due_date',
];

protected function casts(): array {
    return ['due_date' => 'datetime'];
}

public function project() {
    return $this->belongsTo(Project::class);
}

public function creator() {
    return $this->belongsTo(User::class, 'created_by');
}

public function assignees() {
    return $this->belongsToMany(User::class);
}
```

6.3 Buat route resource task

```
Route::middleware('auth')->group(function () {
    Route::resource('tasks', TaskController::class);
    Route::patch('/tasks/{task}/status', [TaskController::class, 'updateStatus'])
        ->name('tasks.status');
});
```

6.4 Implementasi CRUD

Alur controller:

Method	Pekerjaan
index	Tampilkan task yang dapat diakses user.
create	Ambil daftar project user lalu tampilkan form.
store	Validasi input, set created_by=auth()->id(), simpan.
show	Tampilkan detail task.
edit	Tampilkan form dengan data task.
update	Validasi dan update field.

Method	Pekerjaan
destroy	Hapus task setelah otorisasi.
updateStatus	Ubah status todo/in_progress/done.

```
$data = $request->validate([
    'project_id' => ['required','exists:projects,id'],
    'title' => ['required','max:255'],
    'description' => ['nullable','string'],
    'priority' => ['required','in:low,medium,high'],
    'status' => ['required','in:todo,in_progress,done'],
    'due_date' => ['nullable','date'],
]);
$data['created_by'] = auth()->id();
Task::create($data);
```

6.5 Buat view task

- resources/views/tasks/index.blade.php: tabel/kartu task dengan title, project, priority, status, due date.
- resources/views/tasks/create.blade.php: form project, title, description, priority, status, due_date.
- resources/views/tasks/edit.blade.php: form yang sama dengan nilai lama.
- resources/views/tasks/show.blade.php: detail lengkap dan daftar assignee bila tersedia.
- Tombol Delete memakai method DELETE dan @csrf.
- Tombol "Selesai" mengirim PATCH status=done.

6.6 Aturan akses minimum

Jangan biarkan semua user mengedit task orang lain. Minimum untuk tugas kuliah: task dapat diedit jika user adalah creator, owner project, atau anggota project. Implementasi sederhana dapat diletakkan di controller atau Policy.

```
abort_unless(
    $task->created_by === auth()->id()
    || $task->project->owner_id === auth()->id()
    || $task->project->members->contains(auth()->id()),
    403
);
```

6.7 Pengujian Orang 2

Tes	Expected
Create task valid	Task masuk database dan tampil di list.
Title kosong	Validasi gagal.
Edit task	Data berubah.
Mark done	Status menjadi done.
Delete task	Task hilang.
User tidak berhak akses	403.
Deadline kosong	Tetap boleh disimpan.

6.8 Commit dan PR

```
git add .
git commit -m "feat: add task CRUD and task status"
git push -u origin feature/task
```

Definition of Done - CRUD task berfungsi, validasi aktif, status dapat berubah, dan tidak ada

7. Tugas Orang 3 - Project & Team Collaboration

Item	Isi
Branch	feature/project-team
Tujuan	CRUD project, anggota project, dan assign user ke task.
Tabel utama	projects, project_user, task_user.
Selesai jika	Owner dapat membuat project, menambah anggota, dan menentukan assignee task.

7.1 Mulai branch

```
git checkout develop
git pull origin develop
git checkout -b feature/project-team
php artisan make:controller ProjectController --resource
php artisan make:controller ProjectMemberController
php artisan make:controller TaskAssignmentController
```

7.2 Isi model Project

```
protected $fillable = ['name', 'description', 'owner_id'];

public function owner() {
    return $this->belongsTo(User::class, 'owner_id');
}

public function members() {
    return $this->belongsToMany(User::class);
}

public function tasks() {
    return $this->hasMany(Task::class);
}
```

7.3 CRUD project

```
// store
$data = $request->validate([
    'name' => ['required', 'max:255'],
    'description' => ['nullable', 'string'],
]);
$data['owner_id'] = auth()->id();
$project = Project::create($data);
$project->members()->syncWithoutDetaching([auth()->id()]);
```

- index(): tampilkan project yang dimiliki atau diikuti user.
- edit/update/destroy(): hanya owner yang boleh melakukan perubahan penting.
- show(): tampilkan anggota project, task dalam project, dan progress placeholder.

7.4 Tambah dan hapus anggota project

```
// tambah anggota
$request->validate(['user_id' => ['required', 'exists:users,id']]);
abort_unless($project->owner_id === auth()->id(), 403);
$project->members()->syncWithoutDetaching([$request->user_id]);

// hapus anggota
abort_unless($project->owner_id === auth()->id(), 403);
$project->members()->detach($user->id);
```

7.5 Assign user ke task

Pastikan user yang dipilih memang anggota project dari task tersebut.

```
abort_unless($task->project->owner_id === auth()->id(), 403);

$ids = $request->validate([
    'user_ids' => ['array'],
    'user_ids.*' => ['exists:users,id'],
])[ 'user_ids' ] ?? [];

$validIds = $task->project->members()
    ->whereIn('users.id', $ids)
    ->pluck('users.id');

$task->assignees()->sync($validIds);
```

7.6 Route project/team

```
Route::middleware('auth')->group(function () {
    Route::resource('projects', ProjectController::class);
    Route::post('/projects/{project}/members', [ProjectMemberController::class, 'store']);
    Route::delete('/projects/{project}/members/{user}', [ProjectMemberController::class, 'destroy']);
    Route::put('/tasks/{task}/assignees', [TaskAssignmentController::class, 'update']);
});
```

7.7 View yang dibuat

- projects/index.blade.php: daftar project milik/diikuti user.
- projects/create.blade.php dan edit.blade.php.
- projects/show.blade.php: detail project, daftar anggota, daftar task.
- Komponen form tambah anggota menggunakan dropdown user.
- Komponen assign task menggunakan checkbox/multi-select anggota project.

7.8 Pengujian Orang 3

Tes	Expected
Create project	Owner adalah user login.
Tambah anggota	User muncul di members.
Tambah anggota sama dua kali	Tidak duplikat.
Hapus anggota	Relasi terhapus.
Assign anggota ke task	task_user terisi.
Assign non-member	Tidak ikut tersimpan.
Non-owner mengelola anggota	403.

7.9 Commit dan PR

```
git add .
git commit -m "feat: add project membership and task assignment"
git push -u origin feature/project-team
```

Definition of Done - Project memiliki owner dan members yang konsisten, serta task hanya dapat di-assign kepada member project.

8. Tugas Orang 4 - Priority, Deadline & Progress

Item	Isi
Branch	feature/progress
Tujuan	Dashboard progress, prioritas, deadline, filter, ringkasan.
Tabel utama	Menggunakan projects/tasks yang sudah ada; tidak perlu tabel progress.
Selesai jika	User dapat mengetahui berapa task selesai dari total dan melihat prioritas/deadline.

8.1 Mulai branch

```
git checkout develop
git pull origin develop
git checkout -b feature/progress
php artisan make:controller DashboardController
```

8.2 Query progress project

```
$projects = Project::query()
    ->where('owner_id', auth()->id())
    ->orWhereHas('members', fn ($q) => $q->where('users.id', auth()->id()))
    ->withCount([
        'tasks',
        'tasks as done_tasks_count' => fn ($q) => $q->where('status', 'done'),
    ])
    ->get()
    ->map(function ($project) {
        $project->progress = $project->tasks_count > 0
            ? round(($project->done_tasks_count / $project->tasks_count) * 100)
            : 0;
        return $project;
    });
```

8.3 Ringkasan dashboard

Tampilkan minimal empat angka: total task, todo, in progress, dan done.

```
$base = Task::whereHas('project', function ($q) {
    $q->where('owner_id', auth()->id())
    ->orWhereHas('members', fn ($m) => $m->where('users.id', auth()->id()));
});

$total = (clone $base)->count();
$todo = (clone $base)->where('status', 'todo')->count();
$inProgress = (clone $base)->where('status', 'in_progress')->count();
$done = (clone $base)->where('status', 'done')->count();
```

8.4 Deadline dan prioritas

- Pada list task tampilkan badge Low/Medium/High.
- Tampilkan due date dengan format mudah dibaca.
- Jika due_date sudah lewat dan status belum done, beri label Overdue.
- Buat filter query: status, priority, dan urutan due_date.
- Buat section "Deadline terdekat" misalnya 5 task dengan due_date paling dekat.

```
$upcoming = Task::where('status', '!=', 'done')
    ->whereNotNull('due_date')
    ->where('due_date', '>=', now())
    ->orderBy('due_date')
    ->limit(5)
    ->get();
```

8.5 Route dan view dashboard

```
Route::get('/dashboard', [DashboardController::class, 'index'])
    ->middleware('auth')->name('dashboard');
```

- resources/views/dashboard.blade.php: kartu ringkasan angka, progress bar per project, deadline terdekat.
- Gunakan width progress bar dari nilai 0-100; jangan simpan persen ke database.
- Bila total task = 0, progress harus 0%, bukan error pembagian nol.

8.6 Contoh tampilan target

```
Project: JARA Website
Total Task      : 10
To Do           : 2
In Progress     : 2
Done            : 6
Progress        : 60%
Deadline dekat  : "Selesaikan halaman admin" - 12 Sep 2026
```

8.7 Pengujian Orang 4

Tes	Expected
Project tanpa task	Progress 0%.
10 task, 6 done	Progress 60%.
Semua task done	Progress 100%.
Task lewat deadline belum done	Muncul Overdue.
Filter high	Hanya high priority.
Dashboard user A	Tidak menampilkan project yang tidak diikuti user A.

8.8 Commit dan PR

```
git add .
git commit -m "feat: add progress dashboard priority and deadlines"
git push -u origin feature/progress
```

Definition of Done - Persentase benar, tidak ada division-by-zero, deadline/prioritas tampil, dan data dashboard dibatasi sesuai user.

9. Integrasi Seluruh Branch

Setelah masing-masing PR siap, integrasikan ke develop. Untuk mengurangi konflik, gunakan urutan berikut:

18. Merge feature/user-admin ke develop.
19. Merge feature/project-team ke develop.
20. Merge feature/task ke develop.
21. Update feature/progress dari develop, selesaikan konflik bila ada, kemudian merge feature/progress.
22. Jalankan migrasi, build frontend, dan testing bersama.

9.1 Setiap sebelum merge

```
git checkout feature/NAMA-FITUR
git fetch origin
git merge origin/develop
# selesaikan conflict bila ada
git add .
git commit -m "chore: resolve develop merge conflicts"
git push
```

9.2 Setelah semua merge di develop

```
git checkout develop
git pull origin develop
composer install
npm install
npm run build
php artisan migrate:fresh
php artisan route:list
php artisan test
```

Peringatan migrate:fresh - Perintah ini menghapus data lokal. Gunakan hanya selama development/testing, bukan pada database produksi.

10. Menjalankan Hasil Akhir di Localhost

Untuk demo kelompok pada komputer yang sudah memiliki project:

```
git checkout develop
git pull origin develop
composer install
npm install
npm run build
php artisan migrate
composer run dev
```

Laravel 13 menyediakan composer run dev untuk menjalankan development server dan proses frontend yang diperlukan. Alternatif sederhana jika hanya perlu web server:

```
php artisan serve
```

Buka <http://127.0.0.1:8000> atau <http://localhost:8000>.

10.1 Alur demo yang disarankan

23. Login sebagai user biasa.
24. Buat project baru.
25. Tambahkan user lain sebagai anggota.
26. Buat beberapa task dengan prioritas dan deadline berbeda.
27. Assign task kepada anggota.

28. Ubah sebagian task menjadi In Progress dan Done.
29. Buka dashboard dan tunjukkan progress berubah otomatis.
30. Logout, login sebagai admin, lalu tunjukkan daftar user dan fungsi tambah/hapus akun.

11. Push Final ke GitHub dan README

11.1 Merge develop ke main setelah stabil

```
git checkout main
git pull origin main
git merge develop
git push origin main
```

Lebih baik lakukan merge melalui Pull Request develop -> main agar riwayat review terlihat.

11.2 Isi README.md minimum

- Nama project: JARA - Advanced To Do List.
- Deskripsi singkat fitur.
- Nama anggota dan pembagian tugas.
- Kebutuhan: PHP 8.5, Composer, Node/NPM, database.
- Cara instalasi dari clone sampai migrate.
- Cara menjalankan localhost.
- Akun demo admin bila diperlukan (jangan gunakan password pribadi).
- Screenshot halaman utama/dashboard bila diminta dosen.

```
git clone <URL_REPOSITORY>
cd JARA-Advanced-ToDoList
composer install
npm install
copy .env.example .env
php artisan key:generate
php artisan migrate
npm run build
composer run dev
```

.env - Jangan commit file .env karena dapat berisi konfigurasi/password. Commit .env.example dengan nilai contoh yang aman.

12. Jira Board dan Pembagian Issue

Gunakan board Kanban sederhana: To Do -> In Progress -> Review -> Done. Buat satu Epic per modul agar kontribusi anggota terlihat.

Epic	Contoh Issue	Assignee
User & Admin	Register/Login, Admin Middleware, Manage Users	Orang 1
Task Management	Create Task, Edit Task, Delete Task, Change Status	Orang 2
Project & Team	Create Project, Add Member, Assign Task	Orang 3
Progress	Dashboard, Progress %, Priority Filter, Deadline	Orang 4
Integration	Merge, regression test, README, demo localhost	Semua

Setiap issue sebaiknya memiliki acceptance criteria dan link Pull Request GitHub. Saat PR merged dan sudah diuji, pindahkan issue ke Done.

13. Checklist Pengujian Akhir

- ☐ Project bisa di-clone dari GitHub tanpa file lokal tersembunyi.
- ☐ composer install dan npm install berhasil.
- ☐ php artisan migrate berhasil dari database kosong.
- ☐ Register/login/logout berfungsi.
- ☐ Admin tidak dapat diakses user biasa.
- ☐ Admin dapat tambah/hapus user.
- ☐ User dapat membuat project.
- ☐ Owner dapat tambah/hapus anggota project.
- ☐ User dapat membuat/edit/hapus task sesuai izin.
- ☐ Task memiliki priority dan due date.
- ☐ Task dapat di-assign kepada anggota project.
- ☐ Status todo/in_progress/done berfungsi.
- ☐ Progress project dihitung benar.
- ☐ Overdue dan deadline terdekat tampil benar.
- ☐ Dashboard tidak membocorkan project user lain.
- ☐ Tidak ada error 500 selama alur demo.
- ☐ Branch dan commit tiap anggota terlihat di GitHub.
- ☐ README menjelaskan cara menjalankan project.

14. Troubleshooting Umum

Masalah	Solusi cepat
'php' is not recognized	Tutup terminal, buka lagi, lalu cek <code>php -v</code> ; pastikan PATH PHP tersedia.
Class not found setelah pull	Jalankan composer install dan php artisan optimize:clear.
Vite manifest not found	Jalankan npm install lalu npm run build, atau composer run dev.
SQLSTATE / table not found	Periksa .env lalu jalankan php artisan migrate.
Migration conflict	Jangan mengubah migration lama yang sudah dibagikan; buat migration baru untuk perubahan schema.
419 Page Expired	Pastikan form POST/PUT/PATCH/DELETE memiliki @csrf.
403	Periksa middleware/otorisasi dan user yang sedang login.

Masalah	Solusi cepat
Route not found	Jalankan php artisan route:list dan periksa nama/method route.
Git merge conflict	Buka file conflict, pilih isi yang benar, hapus marker <<< ==== >>>, lalu git add dan commit.
Perubahan .env tidak terbaca	Jalankan php artisan config:clear dan restart server.

Ringkasan File yang Paling Mungkin Disentuh

Orang	Folder/File utama
1	app/Models/User.php; AuthController; Admin/UserController; AdminMiddleware; views/auth; views/admin/users; routes/web.php
2	app/Models/Task.php; TaskController; views/tasks; routes/web.php
3	app/Models/Project.php; ProjectController; ProjectMemberController; TaskAssignmentController; views/projects; routes/web.php
4	DashboardController; views/dashboard.blade.php; filter/list UI; routes/web.php

Koordinasi routes/web.php - Karena semua orang mungkin mengubah file route yang sama, conflict kecil sangat mungkin terjadi. Jangan panik; gabungkan seluruh route yang masih diperlukan dan pastikan tidak ada nama route duplikat.

Referensi Singkat

Dokumentasi resmi Laravel 13.x: <https://laravel.com/framework/docs>

Laravel Starter Kits: <https://laravel.com/starter-kits>

Laravel Learning - Basic Authentication: <https://laravel.com/learn/getting-started-with-laravel>

Dokumen ini disusun untuk project kuliah JARA dan berfokus pada implementasi lokal serta workflow GitHub yang sederhana.